

شماره دانشجویی: ۴۰۱۸۳۲۲۳۰

کلی فاضل نیا

تمرین های فصل سوم (لیست ، صف و ارزشیابی عبارت های ریاضی)

تمرین 3-1: لیست را با لیست پیوندی پیاده سازی کرده و الگوریتم های push و pop را بنویسید.

اگر لیست خالی باشد
node *top = NULL
node *nnext; top در اینجا حکم همان start در لیست های پیوندی را دارد.

```
struct node {  
    int data;  
    node *nnext;  
};  
void push (node **top, cntn) {  
    node *t = new node();  
    t->data = cntn;  
    t->nnext = *top;  
    *top = t;  
}
```

الگوریتم push

صفحه جدیدی که می خواهیم اضافه کنیم را به ابتدای لیست پیوندی اضافه می کند.

```
int pop (node **top) {  
    if ((*top) == NULL) {  
        cout << "under flow";  
        return -1;  
    }  
    int n;  
    n = (*top)->data;  
    (*top) = (*top)->nnext;  
    return n;  
}
```

الگوریتم pop

ابتدای لیست پیوندی را حذف می کند و مقدار اولین نود (صفحه) را در مقدار n می ریزد و بهی گرداند (اول شرط را چک می کند تا اگر لیست خالی است ، under flow را برگرداند.)

```
void main ()  
{
```

node *top = NULL; → در ابتدای کار لیست خالی است.

push (&top, 1);

push (&top, 2);

cout << pop (&top);

Parsian cout << pop (&top);
cout << pop (&top);
}

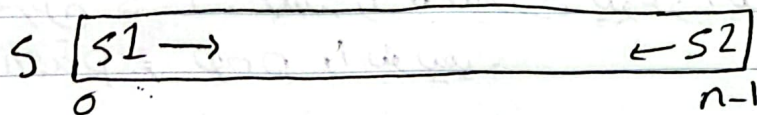
در اینجا دو صفحه (گره) اضافه می کنیم و آنها را چاپ می کنیم. مستحقاً آخرین pop ، عبارت under flow را برمی ما چاپ می کند.


```

if (c == 2) {
    if (*top2 == n) {
        cout << "under flow";
        return -1;
    }
}
return s[(*top2)++];
}

```

تمرین 3-2: فرض کنید می خواهیم دو پشته S1 و S2 را در یک آرایه به طول n پیاده سازی کنیم. الگوریتم های push(S1, n), push(S2, n), pop(S1), pop(S2) را بنویسید.



S آرایه به طول n
پشته S1 و $top1 = -1$ پشته خالی است و از ابتدا شروع کرده و به آخر می رود.
پشته S2 و $top2 = n$ پشته خالی است و از آخر شروع کرده و به ابتدا می رود.
شرط پر بودن: $top1 + 1 = top2$?

```

void push(int s[], int c, int x, int *top1, int *top2)

```

```

{
    if (*top1 + 1 == *top2) {

```

```

        cout << "overflow";
    }

```

```

    if (c == 1) {
        s[++(*top1)] = x;

```

```

    } else s[--(*top2)] = x;
    }

```

الگوریتم push
در ابتدا شرط پر بودن چک می شود و اگر پر نباشد، اگر c یعنی شماره پشته 1 یا 2 باشد، به ابتدای آن اضافه می کند و اگر 2 بود به پشته دوم و ابتدای آن را اضافه می کند.

```

int pop(int s[], int c, int *top1, int *top2)

```

```

{
    if (c == 1) {

```

```

        if (*top1 == -1) {

```

```

            cout << "underflow";
            return -1;

```

```

        } return s[(*top1)--];
    }

```

الگوریتم pop: در ابتدا c یعنی شماره

پشته مورد نظر دارد و پشته و طبق شماره آن به خالی بودن

هر کدام چک می شود و اگر خالی نبود، مقدار ابتدای

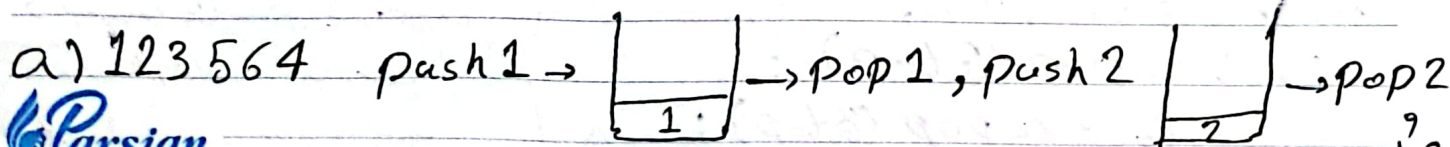
هر کدام را می خواهیم به می گرداند.

تمرین 3-3: یک پشته خالی به همراه اعداد 1 تا 6 در ورودی داده شده است. اتمال زیر به روی پشته قابل انجام است.

Push: کوچک ترین عدد ورودی را برداشته و وارد پشته می کند.

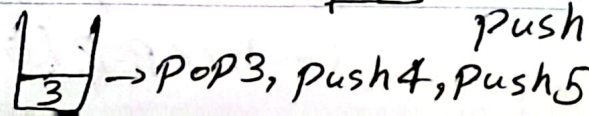
Pop: عنصر بالای پشته را در خروجی نوشته و سپس آن را از پشته حذف می کند. بررسی کنید که آیا می توان هر کدام از مولد زیر را با ترتیب متفاوتی از اتمال پشته بدست آورد.

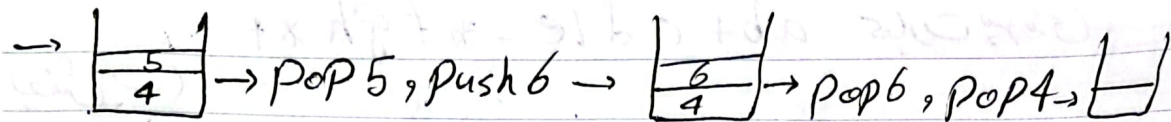
a) 123564 b) 215346 c) 431265



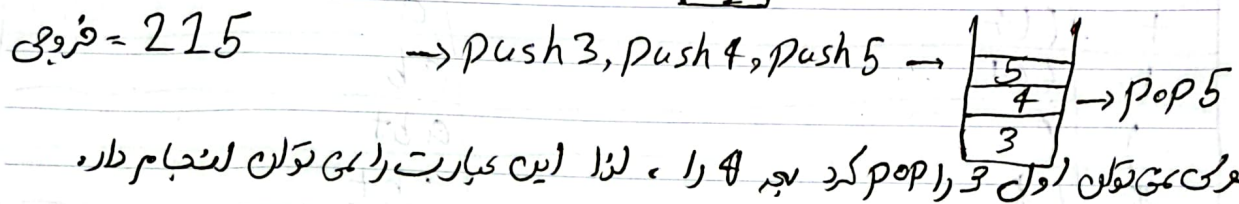
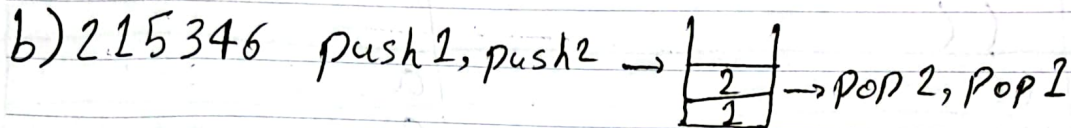
خروجی = 1 2 3 5 6 4

صفحه دوم

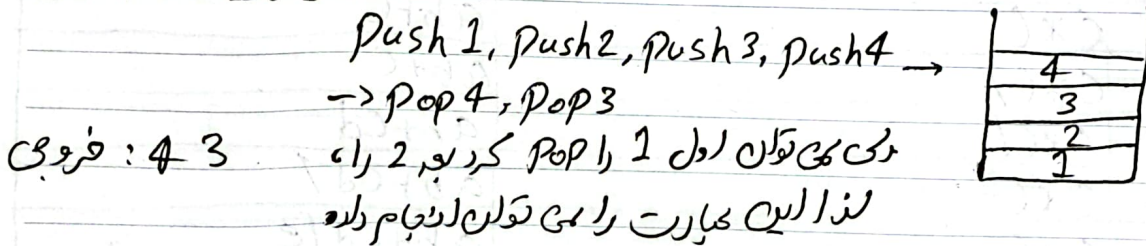




پس عبارت a را می توان انجام داد.



c) 431265



تمرین 3-4: عبارت میانه و نهایی زیر را در نظر بگیرید.

enfix: $(a+b) * (c/d-e) + f * g * h$

الف) معادل پیشوندی و پسوندی آن را بدون استفاده از پرانتز بنویسید.

ب) معادل پسوندی آن را با استفاده از پرانتز بنویسید.

$$(a+b) * (c/d-e) + f * g * h$$

$$ab+ ((cd)-e) fg * * h$$

ردیف دوم

$$(cd/e-)$$

$$fg * h *$$

$$(* + fg * h *)$$

$$(ab + * cd/e -)$$

$$\rightarrow * fg * h * +$$

که همان نتیجه را می دهد.

$$\rightarrow ab + cd/e - * + f * g * h$$

Parstian

$$ab + cd/e - * fg * h * +$$

$$* f +$$

$$* fg * + * h$$

ردیف اول

$$* fg * h * *$$

صفحه سوم

$$\begin{array}{|c|c|c|} \hline n & (+ * & \\ \hline & & \{ ab+cd \mid e-*fg^*h \\ & & ab+cd \mid e-*fg^*h * + \\ \hline \end{array}$$

→ عبارت سوزنی $ab+cd \mid e-*fg^*h * +$

(بخش ب)

مکان فعلی	محتوای سسته	فروبی
(	((	
a	((a	a
+	(((+	a
b	(((+b	ab
)	((	ab+
*	((*	ab+
(	((*(	ab+
c	((*(c	ab+c
/	((*(/	ab+c
d	((*(d	ab+cd
-	((*(-	ab+cd/
e	((*(-e	ab+cd \mid e
)	((*(	ab+cd \mid e-
g	((*+g	ab+cd \mid e-g*
f	((*+f	ab+cd \mid e-*f
*	((*+*	ab+cd \mid e-*f
g	((*+*	ab+cd \mid e-*fg
*	((*+*	ab+cd \mid e-*fg*

مرحله 3-6: عبارت سوزنی $ab+cd \mid e-*fg^*h * +$ را با استفاده از سسته به عبارت میانبر تبدیل کنید.

مکان فعلی	محتوای سسته در ابتدا	محتوای سسته	اول	a	b	*	c	\	d	e	f	*	h	+	*
				a	a	(a*b)	(a*b)	((a*b) \)	((a*b) \ d)	((a*b) \ d e)	((a*b) \ d e f)	((a*b) \ d e f *)	((a*b) \ d e f * h)	((a*b) \ d e f * h +)	((a*b) \ d e f * h + *)


```
front = Null; *rear = Null;
```

```
else
```

```
{ n = (*front) -> data;
```

```
(*front) = (*front) -> next;
```

```
} return x;
```

تمرین 3-8: صف را یا لیست پیوندی پیاده سازی کرده و الگوریتم های enqueue و dequeue را بنویسید.

```
struct node {
```

```
int data;
```

```
node *next;
```

```
}
```

```
void enqueue (node **front, node **rear, int n)
```

```
{
```

```
node *t = new node();
```

```
t -> data = n
```

```
t -> next = Null
```

```
if (*rear == Null)
```

```
{ *front = t; *rear = t; }
```

```
else {
```

```
(*rear) -> next = t;
```

```
*rear = t
```

```
}
```

```
int dequeue (node **front, node **rear) { int n;
```

```
if (*front == Null)
```

```
{ cout << "underflow", return -1; }
```

```
if (*front == *rear)
```

```
{ n = (*front) -> data
```

ادامه در صفحه بعد

تمرین 3-9: با داشتن دو صف 91، 92، یک رشته را پیاده سازی کنید. صف

```
void Push (Queue *q1, int n)
```

```
{
```

```
enqueue(q1, x);
```

```
}
```

```
int pop (Queue *q1, Queue *q2)
```

```
{
```

```
int n;
```

```
while (q1 -> front != q1 -> rear)
```

```
{ x = dequeue(q1);
```

```
if (q1 -> front == q1 -> rear)
```

```
return x;
```

```
else
```

```
enqueue(q2, n);
```

اول چک می کند اگر لیست خالی باشد، عبارت underflow چاپ شود، در غیر این صورت اگر فقط یک عنصر داشته باشد، همان عنصر را چاپ می کند و لیست را خالی می کند. غیر از این (این صورت اولین) که را چاپ کرده و گره بعدی را جای آن قرار می دهد.

الگوریتم enqueue

این الگوریتم گره مورد نظر را

t را به آخر باید اضافه کند

لذا اول شرط چک می کند

صف خالی است یا نه و اگر خالی

بود، گره مورد نظر را اضافه می کند

(یا گره لیست پیوندی تشکیل می دهد)

و اگر خالی نبود، گره t را به آخر

لیست پیوندی اضافه می کند.

الگوریتم dequeue

این الگوریتم

یک عنصر را از

ابتدای لیست پیوندی

حذف می کند.

الگوریتم Push

با Push کردن یک عنصر در واقع آن

را به آخر صف q1 اضافه می کنیم یعنی همان enqueue.

الگوریتم pop

الگوریتم pop باید آخرین

عنصر رشته حذف شود،

ولی dequeue در صف

اولین عنصر حذف را

حذف می کند. لذا به

دو رشته نیاز داریم.

ادامه در صفحه بعد


```
while (q2 → front != q2 → rear)
{
```

```
    n = dequeue(q2);
    enqueue(q1, x);
}
```

```
}
```

برای این شکل همه عناصر قبل از صفه مورد نظر را حذف کرده و در صف $q2$ می‌ریزیم و زمانی که صفه مورد نظر را حذف کردیم، تمامی عناصر در $q2$ را به $q1$ می‌گردانیم. در حلقه اول یک می‌شود اگر $q1$ تنها یک صفه دارد، همان dequeue شود، در غیر این صورت تا زمانی که یک صفه باقی بماند، تمامی عناصر به $q2$ اضافه شوند. در حلقه دوم تا زمانی که تنها یک صفه باقی بماند (تمامی صفه‌های که می‌خواهیم pop کنیم)، تمامی عناصر قبل از $q2$ پاک خواهند شد و به $q1$ enqueue اضافه خواهند شد.

تمرین 3-10: با داشتن دو پشته $S1$ و $S2$ ، یک صف را پیاده‌سازی کنید.

```
void enqueue (stack *s1, int n)
```

```
{
```

```
    push(s1, n);
```

```
}
```

```
int dequeue (stack *s1, stack *s2)
```

```
{
```

```
    int n;
```

```
    while (s1 → top != -1)
```

```
    {
```

```
        n = pop(s1);
```

```
        if (s1 → top == -1)
```

```
            return n;
```

```
        else
```

```
            push(s2, n);
```

```
    }
```

```
    while (s2 → top != -1)
```

```
    {
```

```
        n = pop(s2);
```

```
        push(s1, n);
```

```
    }
```

الگوریتم enqueue

با اضافه کردن صفه به صف در واقع به آخر آن اضافه می‌شود

که با push کردن در $S1$ می‌توان آن را انجام داد.

الگوریتم dequeue :

برای حذف یک صفه از پشته،

اولین صفه‌ای که به $push S1$

شده باید حذف گردد. لذا تمامی

عناصر بالای اولین صفه

pop شده و به پشته $S2$ اضافه

می‌شود. زمانی که به صفه مورد

نظر رسیدیم و آن را pop کنیم

تمامی عناصر موجود در $S2$ را

به $S1$ می‌گردانیم.